

First Gear Bank v1 Delivery Roadmap

This roadmap divides the locked v1 specification into independently reviewable implementation plans. The split is deliberate: the financial core, physical inventory settlement, network/UI layer, and branch/world-generation systems have different failure modes and can each be accepted without holding the entire program in one working context.

The authoritative behavior remains `docs/superpowers/specs/2026-09-08-first-gear-bank-v1-design.md`. A phase plan may choose concrete class names and file boundaries, but it may not weaken or rebalance that specification.

Delivery Order

Phase	Executable plan	Primary specification coverage	Exit gate
1	2026-09-08-first-gear-bank-v1-foundation.md	Foundational portions of sections 1-6, 18, 20.2-20.3, 21, 22, and the relevant section 23 tests	A pure .NET core, safe package pipeline, immutable ledger/coordinator, sectioned persistence, and settlement-recovery feasibility spike all pass review.
2	2026-09-08-first-gear-bank-v1-financial-core.md	Sections 7-9	Both clocks, Constant/CIR rates, RNG domains, shocks, and rusty/temporal accrual pass deterministic and statistical tests.
3	2026-09-08-first-gear-bank-v1-physical-settlement.md	Section 10 and settlement portions of sections 20-21	Denominations and inventory operations are atomic during live execution, and every hard-crash state is either proved safe or quarantined.
4	2026-09-08-first-gear-bank-v1-certificates-liquidity.md	Section 12	CD quotes, contracts, maturity, the exact accrued-liability index, and liquidity adjustments pass deterministic tests using genuinely funded accounts.
5	2026-09-08-first-gear-bank-v1-identity-transfers.md	Sections 5, 11, and 19	The current-epoch per-world registry, transfers, durable notices, and administrative corrections pass the complete identity matrix.
6	2026-09-08-first-gear-bank-v1-protocol-statements.md	Sections 13 and 20.1	Authenticated sessions, bounded DTOs, pagination, live/printed statements, scraps, fuel, and compost pass protocol and engine tests.
7	2026-09-08-first-gear-bank-v1-charter-banker-gui.md	Sections 14-15 and 17	Charter protection/lifecycle, Banker entity and replacement, and isolated player GUIs pass engine and multiplayer tests without Harmony.
8	2026-09-08-first-gear-bank-v1-natural-branches-release.md	Sections 16 and 22-26	Natural generation/backfill, optional integrations, diagnostics, load tests, all acceptance scenarios, release packaging, and rendered QA pass.

Only the first plan is authored now. Each later executable plan is written after its prerequisites are real and reviewed, so it can cite the actual interfaces, schema version, API-spike evidence, and test fixtures produced upstream.

Dependency Graph

```
Phase 1: evidence, contracts, journal, coordinator, sectioned persistence
  -- Phase 2: financial clocks, rates, shocks, and cash accrual
    -- Phase 3: denominations and physical settlement
      | -- Phase 4: CDs and exact liquidity
      |   -- Phase 5: identity, transfers, notices, corrections
      |     -- Phase 6: protocol and statements
      |       -- Phase 7: Charter, Banker, and GUI
      |         -- Phase 8: natural branches and release
    -- branch API evidence -----^
```

Phase 3 deliberately precedes CDs and transfers so those systems can be tested with accounts funded through the real deposit path rather than privileged test seeding. Protocol types are separated from the GUI; the GUI joins the Banker and Charter phase because its valid session depends on a live assigned Banker and branch.

Contracts Frozen in Phase 1

Phase 1 owns the cross-cutting decisions that every later plan must reuse:

- signed six-decimal `BankUnits`, ties-to-even conversion, and the `int.MaxValue`-gear customer cap;
- `Currency`, hidden player keys, operation/command/request identifiers, and normalized financial instants;
- immutable, append-only, independently balanced journal records;
- one serialized server-side coordinator and all-or-nothing revision publication;

- request-scope sequencing, digest matching, bounded response caching, and permanent request/automatic-operation barriers;
- deterministic projection replay and the rule that caches never outrank the journal;
- explicit verified/unavailable domain slots, gated mutation reads, and audited quarantine transitions without fabricated empty authority;
- framed, independently checksummed, versioned persistence sections with a migration registry, opaque quarantined-section pass-through, and whole-envelope blocking for unknown container-v1 kinds;
- an authenticated, bounded physical-settlement recovery plan containing the complete predeclared Bank drafts, original request facts, and exact inventory slot deltas—not merely a non-reconstructive digest; and
- engine-neutral ports for time, world blobs, logging, inventory settlement, sessions, topology, and branch operations.

The foundation freezes envelope framing, section identity, journal representation, key formats, and quarantine rules. It does not falsely declare every later section complete. Every additive phase advances its independently versioned section through a deterministic migration, and the release gate exercises migration from every committed development schema.

The envelope uses per-section checksums so registry corruption can isolate name services without taking healthy finance offline, and branch corruption cannot invalidate the journal. Corruption in the journal or indispensable market state still places finance in read-only quarantine as the specification requires.

Vintage Story's `GameWorldSave` hook is a pre-write staging event, not a durable-save acknowledgment. The live host therefore restages the current immutable snapshot on every event and never logs a disk-durability claim the API cannot make.

Locked Implementation Interpretations

These interpretations follow the v03 text and prevent later plans from choosing incompatible persistence shapes:

- `FinancialInstant` is normalized as completed financial months plus a decimal fraction in $[0, 1)$. Whole-month CD tenors therefore add exactly and do not depend on binary floating-point equality.
- A positive administrative correction does not open a nonexistent account. Section 5 says only a successful physical deposit or received transfer creates the banking relationship.
- An initially individual, unacknowledged notice retains its stable notice identity if the recipient disconnects. It is retried as that individual notice and is not silently merged into a later offline aggregate.
- Ordinary configuration loading never rewrites a valid JSONC file. First creation and an explicit First Gear Bank-owned save emit the canonical commented form; a third-party manager's valid comment-free output remains untouched.
- A player recovery capsule is accepted only when its world binding, player binding, settlement identifier, direction, manifest, before/after fingerprints, complete predeclared journal inputs, original request digest/command/timestamp, exact inventory slot deltas, operation identifiers, and authentication tag verify. Missing, digest-only, or unverifiable evidence means quarantine, never an inferred debit, credit, item removal, or item grant.
- Optional integrations do not gate native operation. Better Loot, Better Loot Plus, Config Lib, and Integrated Mod Manager adapters remain disabled with one bounded warning until their exact installed identifiers and public APIs are verified.

Evidence Gates

The local 1.22.7 reference has already verified the core lifecycle, save-blob API, authenticated server-player events, typed networking, entity/block/item registration, custom GUI primitives, commands/permissions, vanilla currency codes, and reusable crafting ingredients via `consume: false`.

The following engine-facing work remains gated by focused spikes:

1. Prove the strongest recovery evidence available across a world-save blob and player-inventory serialization. The public API exposes neither a cross-blob transaction nor an ordering guarantee.
2. Prove public interception for every Charter-protected player modification, including chiseling, rotation/wrench use, furnishing pickup, and multiblock effects. No Harmony fallback is preapproved.
3. Choose between API-only `EntityHumanoid` and `VSSurvivalMod`'s `EntityDressedHumanoid`, then prove rendering, navigation, interaction, death, and unload behavior without inheriting an ordinary trader inventory.
4. Prove stable vanilla generated-trader identity, visit/load hooks, and crash-safe structure placement before natural branches are coded.
5. Verify exact Better Loot, Better Loot Plus, Config Lib, and Integrated Mod Manager identifiers/APIs from installed or authoritative artifacts before an adapter is enabled.
6. Prove an exact accrued-rusty-liability index whose checkpoint cost does not depend on all accounts. In general, `sum(round(f * balance_i))` cannot be recovered from `sum(balance_i)` alone, so a scalar aggregate is insufficient. The CD/liquidity plan cannot freeze its projection schema until this proof and its adversarial tests pass.

If a required public hook fails, implementation stops at that boundary and returns the evidence to the user. It does not silently narrow the behavior or add Harmony.

Program Completion

The mod is complete only after Phase 8 traces all 26 specification sections and all 15 acceptance criteria to passing automated or documented in-world tests. A successful `dotnet build` alone is never treated as gameplay acceptance.